AI Forward Deployed Engineer

The AI Architect & FDE Course



3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe the role of the AI Forward Deployed Engineer
- [x] Build agents with LangChain and choose the right model for a task
- [x] Use evaluations and observability in Langfuse to drive agent design decisions
- [x] Give an agent long-term memory with Agentic RAG over MCP

By the end of today, you'll be able to:

- [ ] Create agent workflows that are durable and ready for production
- [ ] Build stateful agents with LangGraph and Supabase
- [ ] Put a human in the loop for high-stakes decisions
- [ ] Recover from crashes, keeping progress & preventing double payment

Recap

Agents can search for the knowledge they need



Agentic RAG
Semantic search driven by an agent's tool



Model Context Protocol
A standard way to use someone else's tool



Long-term memory achieved through tools
Including a semantic search tool; i.e. agentic RAG



Retrieval should be evaluated
Score the search as well as the final answer

Durable execution

A run that can stop part way and carry on from where it left off



Agents run for a long time

A queue of claims can take hours



Things fail along the way

Rate limits, timeouts and restarts



People take time to reply

A handler might approve tomorrow



Servers come and go

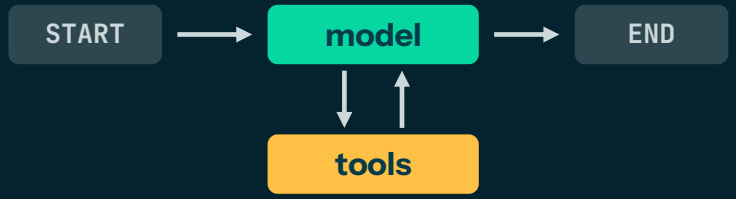
Cloud runtimes stop idle sessions

Introducing LangGraph

A low-level framework and runtime for long-running, stateful agents

Already in use since Week 1

LangChain's create_agent builds this graph behind the scenes



What's different

Graphs you design yourself
Durable execution
Human in the loop
Streaming and memory

Traction

43M downloads last month
42k stars on GitHub
Used by Klarna, Uber and J.P. Morgan

Similar frameworks

OpenAI Agents SDK
Google ADK
CrewAI
Pydantic AI

Today we use two of its core features: durable execution and human in the loop

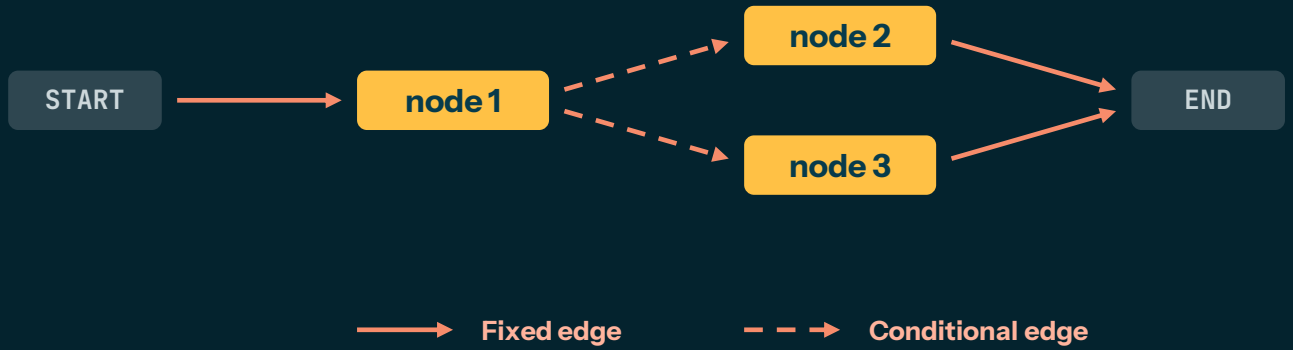
State, nodes and edges

Every LangGraph graph is built from these three parts

State
What the graph keeps as it runs, saved every step

Nodes
Functions that take the state and return what they change

Edges
Which node runs next, fixed or chosen by a function



Building the graph by hand

This code builds the claim graph on the right

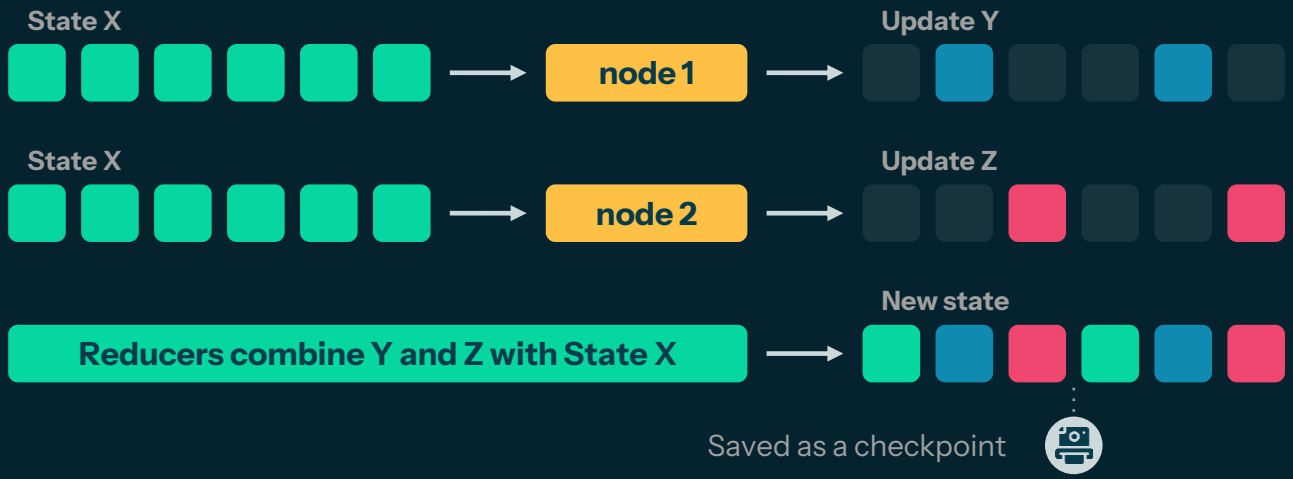
```
builder = StateGraph(ClaimState)
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode([calculate_payout]))
builder.add_node("decide", decide)
builder.add_node("approval", approval)
builder.add_node("pay", pay)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", after_agent)
builder.add_edge("tools", "agent")
builder.add_conditional_edges("decide", after_decide)
builder.add_edge("approval", "pay")
builder.add_edge("pay", END)
```



Also: a reducer is a function that tells LangGraph how to combine a new state with the existing state.

State management

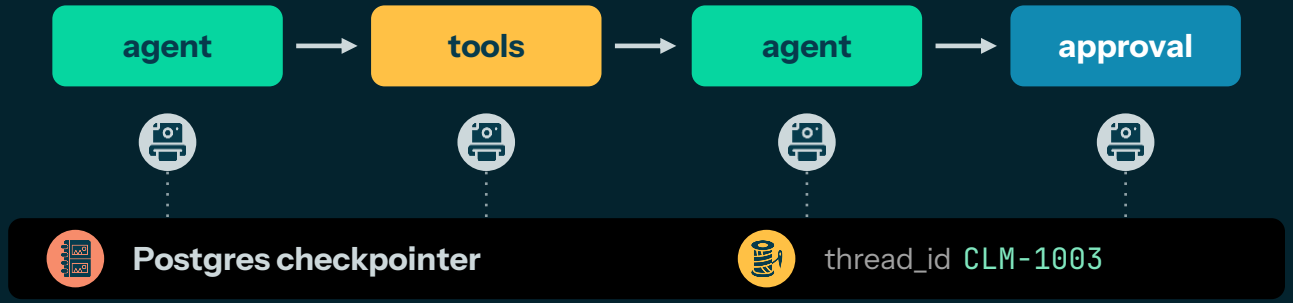
LangGraph combines updates from nodes that run at the same time



LangGraph calls one round like this a super-step (often just "step" in its code)

Checkpoints and threads

LangGraph saves the graph's state after every step




```
checkpointer = PostgresSaver(connection)
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "CLM-1003"}}
graph.invoke(claim, config, durability="sync")

graph.invoke(None, config)    # carry on after a crash
```

Never pay twice

After a crash, the step that was running starts again



An idempotency key on every action

The claim ID makes a second payment for the same claim do nothing



Check the system of record

Ask the database whether the claim has already been paid



Save the state before moving on

With `durability="sync"` each checkpoint is written before the next step



Test it by stopping the run

Kill it part way, run it again and count the payments

Pausing for a human

interrupt() pauses the graph until someone answers

```
def approval(state: ClaimState):
    answer = interrupt({"payout": state["payout"]})
    return {"approved_by": answer["handler"]}

# Later, in any process, when a handler approves
graph.invoke(Command(resume={"handler": "Dana Whitfield"}),
             config)
```



The wait can last days

The state waits in the checkpointer for the answer



The node runs again

Code before interrupt() runs again on resume



Pay in a node of its own

The payment runs once, after the approval

Designing the human's step

Decide when to ask, what to show and what to record



When to ask

Above a limit, when the agent is unsure, before anything irreversible



What the handler sees

The decision, the amount and the reasons with the policy section



What they can do

Approve, reject or change the amount, with a note for the agent



What gets recorded

Who decided, when, and what they were shown

Over-use of human-in-the-loop can lead to decision fatigue

Supabase

Hosted Postgres for UnderCover's records and the agent's state



A Postgres database in the cloud

A free project needs no card and pauses after a week without use



Connect through the session pooler

Works with LangGraph over IPv4 and allows 15 connections at once



Claude Code can look inside

The Supabase MCP server, read-only and limited to one project



More than a database

Sign-in, file storage and realtime updates when you need them

Today's lab

In labs/week2day3.ipynb and its free twin



Supabase

Create a project and connect from the notebook

From create_agent to a graph

Draw the agent we know, then build the same shape by hand

Pause for approval

A claim over $1,000 waits until you approve it

Crash and carry on

Restart the kernel and resume the claim from Supabase

Upgrading the product

When a handler signs off, the agent carries on with the claim



Supabase
Records and checkpoints

Also in this build: the payout tool and tracing in Langfuse

Building it with Claude Code

The change is written up in labs/week2day3_product_DURABLE.md

1. **Copy the brief into docs**
   `cp labs/week2day3_product_DURABLE.md docs/DURABLE.md`
2. **Let Claude Code look at the database**
   Run the claude mcp add command the lab prints for you
3. **Ask Claude Code for a plan**
   Please plan durable state in docs/DURABLE.md
4. **Let it build and commit each phase**
   Please implement the plan for durable state
5. **Or check out my version**
   `git checkout week2day3-start` or `git checkout week2day3-end`

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer AI ENGINEER

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



53% Complete